Student Name: Om Kasvala

Enrollment Number: 92410133005

Division/Semester: 3EK2 / Semester 6

Course Assignment: TASK 3 - Project Requirement Documentation

Project Title: DeskBot: An AI-Powered Cloud Interactive Assistant

Requirements Specification Document

1. Functional Requirements (What the System Must Do)

FR1: Posture Detection via Camera

- **Priority:** High
- **Description:** The system's software must use the laptop webcam to track the user's upper body and detect if they are sitting straight or slouching.
- **Acceptance Criteria:** The system successfully classifies the user's position as "Good Posture" or "Slouching" based on camera input.

FR2: Cloud Message Routing

- **Priority:** High
- **Description:** The laptop software must send a real-time message through a cloud service (AWS IoT) whenever a change in posture is detected.
- **Acceptance Criteria:** The cloud system receives the message from the laptop and successfully forwards it to the DeskBot hardware.

FR3: Hardware Visual Feedback

- **Priority:** High
- **Description:** The DeskBot hardware must change the expressions on its OLED screens (e.g., from happy to sad) when it receives a "slouching" alert from the cloud.
- **Acceptance Criteria:** The OLED screens update to the correct face within 2 seconds of the user slouching.

FR4: Touch Sensor Interaction

- **Priority:** Medium
- **Description:** The DeskBot must have a physical touch sensor that, when tapped, changes the screen expression and plays a sound on the buzzer.
- **Acceptance Criteria:** Tapping the sensor immediately triggers the "shy/scanning" face and a brief beep from the buzzer.

2. Non-Functional Requirements (How the System Should Perform)

Performance:

- **Hardware Response Time:** The DeskBot must process incoming cloud messages and update the OLED display in less than 2 seconds to feel interactive.
- **Software Efficiency:** The camera tracking script running on the laptop should consume minimal CPU power so it does not slow down the user's normal work applications.

Reliability:

- **Uptime:** The ESP8266 microcontroller must maintain a stable Wi-Fi connection to the cloud for at least 8 to 10 hours (a standard working day) without needing to be restarted.
- **Fault Tolerance:** If the Wi-Fi disconnects, the DeskBot should automatically try to reconnect without crashing.

Security:

- **Data Privacy:** The camera video feed must only be processed locally on the user's laptop. No images or videos should be saved or uploaded to the cloud (only basic text alerts like "status: slouching" are sent).
- **Connection Security:** The connection between the hardware and the AWS Cloud must be secured using standard IoT certificates.

Usability:

- **Clear Interface:** The visual expressions on the OLED screens must be easy for the user to understand without needing a manual (e.g., sad eyes clearly indicate poor posture).
- **Setup:** The device should simply plug in via a USB cable for power and begin working automatically.

3. System Architecture (How the System is Structured)

The DeskBot system is divided into three main layers: the Software (Input), the Cloud (Routing), and the Hardware (Output).

3.1 System Components

- **Input Layer (Laptop/PC):** A Python script runs locally on the user's computer. It uses the webcam and an open-source library (MediaPipe) to track body points.

- **Cloud Layer (AWS IoT Core):** Acts as the middleman. It uses the MQTT protocol to securely pass simple text messages back and forth between the laptop and the physical robot.
- **Hardware Layer (DeskBot Node):** A physical device sitting on the desk built with an ESP8266 NodeMCU microcontroller.

3.2 Electronics Hardware Details

- **Microcontroller:** ESP8266 (NodeMCU 1.0) - Acts as the brain of the physical robot and handles Wi-Fi.
- **Display:** Two 0.96-inch I2C OLED screens to display the eyes and expressions.
- **Input Sensor:** TTP223 capacitive touch sensor for physical user interaction.
- **Audio Output:** A 5V passive buzzer to play notification chirps and tones.
- **Power Supply:** Powered by a standard 5V USB cable plugged into the laptop or a power bank.

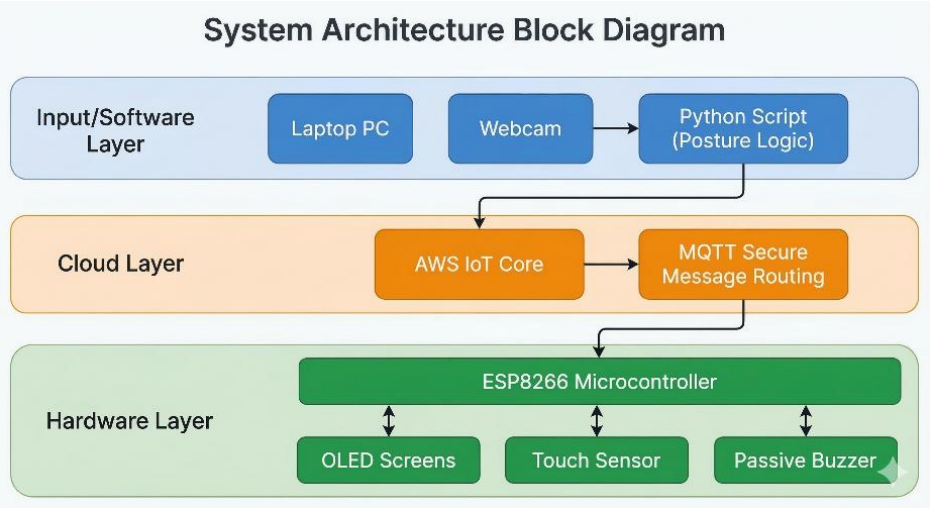
3.3 Technology Stack

- **Machine Learning/Vision:** Python, OpenCV, and MediaPipe.
- **Cloud Platform:** Amazon Web Services (AWS IoT Core) using MQTT protocols.
- **Hardware Programming:** C++ using the Arduino IDE (with Adafruit GFX libraries for screen drawing).

3.4 Data Flow and Logic

1. The Python script looks at the webcam feed and calculates the angle of the user's shoulders and neck.
2. If the angle shows the user is slouching, Python sends an MQTT message `{"posture": "bad"}` to AWS IoT Core.
3. The ESP8266, which is always listening to AWS, receives this message.
4. The ESP8266 runs an `if` statement: if the message is "bad", it commands the OLED screens to draw a sad face and triggers the buzzer to beep, reminding the user to sit up.

Block Diagram (Hardware Connections):



High-Level Data Flow Diagram (DFD

